

Supplément — Vitesse, coût, hallucination : ce qui a été fait, ce qui est mesuré, ce qui est proposé

Document complémentaire au dossier de conception et aux livrables Chantier 1/2, préparé pour le formateur.
Distinction stricte tenue dans tout le document :  **FAIT ET LIVRÉ** (dans le dépôt, testé, fusionné) vs  **PISTE, TESTÉE MAIS NON INTÉGRÉE** (mesurée séparément, aucune ligne du dépôt Mardik modifiée). Date : 23/09/2026.
Dépôt : Ganda15/mardik-api-mlops_v2 , branche main .

Sommaire

1. Ce qui a été réduit, pour de vrai, dans le dépôt livré	2
1.1 Latence — de 11,5 s à moins de 4,2 s sur le contrat le plus lourd	2
1.2 Le vrai modèle, prouvé par la chaîne elle-même	2
1.3 Ce qui reste un vrai manque, dit sans détour	2
2. Pistes explorées ce soir pour réduire encore coût, vitesse, hallucination	3
2.1 Le principe testé : vérification, pas remplacement	3
2.2 Le protocole, identique pour les deux outils (comparaison honnête)	3
2.3 Résultat — TypeSafe (Jev), un service payant à l'appel	3
2.4 Résultat — Laya, un modèle local et gratuit	3
2.5 Comparaison et lecture honnête	4
3. Si le formateur valide d'aller plus loin	4
 Glossaire	4

1. Ce qui a été réduit, pour de vrai, dans le dépôt livré

1.1 Latence — de 11,5 s à moins de 4,2 s sur le contrat le plus lourd

Constat de départ : le budget client est P95 < 8 s. Le 23/09 en fin d'après-midi, le contrat le plus long (c12, 40 pages) mettait **11,45 s** sur une requête isolée réelle — au-dessus du budget.

Deux causes trouvées en mesurant, pas en devinant — un appel de section identique comparé point par point :

Sonde	Direct vers Azure	Via le chemin de service (avant correction)
Un appel de section	1,2 – 1,5 s	3,9 – 7,4 s
23 appels en parallèle (la vague de c12)	max 3,19 s	max 6,66 s

Cause : deux endroits du code ouvraient une **connexion HTTP neuve à chaque appel** (une poignée de main TLS et une résolution DNS à chaque fois) au lieu d'en réutiliser une seule — d'abord dans le client du modèle (`app/llm_client.py`), puis dans le proxy de dérive fourni (`ops/drift_proxy.py`).

Corrections livrées (un seul client HTTP réutilisé à chaque endroit, plus un délai borné de 20 s et une reprise sur erreur transitoire, plus les sections voisines regroupées pour réduire le nombre d'appels) :

Mesure	Avant (22–23/09)	Après (23/09 soir)
P95 du gate, 12 contrats	7 046 ms	4 386 – 7 608 ms sur 11 passages réussis ; 3 passages refusés (8,7 à 12,8 s)
Coût moyen par analyse	0,030 €	0,019 € (ancien modèle à tarif unique) → 0,033 € au tarif catalogue , brique 23, \$1.3
c12, requête isolée (8 mesures réelles)	11 450 ms	2 753 – 4 197 ms
c07, requête isolée	8 800 – 9 800 ms	3 811 ms
Rappel / précision	1,000 / 0,950	1,000 / 0,948 – 0,988 (14 passages du gate réel ; rappel inchangé)

Preuves : `preuves\2026-09-23-latence-ou-passent-les-secondes-sondes.txt` , `preuves\2026-09-23-c12-chemin-de-service-apres-proxy-partage.txt` , `preuves\2026-09-23-gate-reel-brique19-regroupement-essais1.txt` . Commits `3247907` , `6b322f7` .

1.2 Le vrai modèle, prouvé par la chaîne elle-même

Le gate de la chaîne CI tourne en simulation (`MOCK=on` , gratuit) : il prouve la mécanique, pas les vrais chiffres. Un second gate, **manuel, sur le vrai modèle Azure**, a été ajouté (`gate-reel.yml`) : il pose un tag `eval-ok/<commit>` si le vrai gate passe, et la publication peut exiger ce tag avant de livrer.

Exécuté trois fois pour de vrai ce soir (≈ 0,23 € chaque fois) : PASSE à chaque fois. La dernière version publiée sur `main` (`v2.3.2`) porte **le vrai nom du modèle (`gpt-5.4`) et sa vraie signature de dérive** (médiane 0,9917, part de scores bas 0,0, sur 12 analyses) — plus le manifeste générique `modele-ci` . Preuve : `preuves\2026-09-23-manifeste-publie-par-la-chaîne-v2.3.2.json` .

1.3 Ce qui reste un vrai manque, dit sans détour

Le coût annoncé le soir (0,019 €) reposait sur un **prix par jeton unique, hérité de la v1**, jamais confronté au tarif du modèle. **Corrigé le 23/09 à 21 h (brique 23)** : deux tarifs (entrée 0,0025 \$/1k, sortie 0,015 \$/1k — la sortie coûte 6× l'entrée), lus sur une liste publique (OpenRouter, qui relaie le catalogue OpenAI), et le

partage entrée/sortie mesuré dans chaque analyse. Gate réel après correction : **0,033 € par analyse en moyenne, 0,088 pour le contrat de 40 pages** — la cible du CTO (0,15 €) reste tenue, marge ×1,7. Ce qui reste à faire, dit tel quel : confirmer sur le portail Azure que la ressource est bien facturée au tarif catalogue (normalement oui) ; jusque-là, « 0,033 € » est un coût **au catalogue**, pas une ligne de facture.

2. Pistes explorées ce soir pour réduire encore coût, vitesse, hallucination

Rien de cette section n'est dans le dépôt Mardik. Deux outils externes ont été testés séparément, sur des données réelles du projet, sans toucher une seule ligne de `app/` , `ops/` ou `eval/` . La question posée : peuvent-ils vérifier après coup qu'une clause extraite par le modèle est réelle et bien typée — une seconde paire d'yeux, indépendante, avant de faire confiance au score de confiance ?

2.1 Le principe testé : vérification, pas remplacement

Ni l'un ni l'autre outil ne peut **écrire** de texte (l'un et l'autre le disent explicitement dans leur documentation) — ils ne peuvent donc pas remplacer l'étape d'extraction elle-même. Ce qui a été testé est un rôle différent : après que le modèle a proposé une clause avec son extrait littéral, poser une question fermée et indépendante — « cet extrait apparaît-il mot pour mot dans le texte, et décrit-il vraiment ce type de clause ? » — pour attraper une hallucination **après** qu'elle s'est produite, à un coût quasi nul.

2.2 Le protocole, identique pour les deux outils (comparaison honnête)

Un vrai appel au pipeline v2 sur un vrai contrat (c02, coût réel 0,0054 €) a produit **6 clauses réelles** avec leurs extraits littéraux. Deux cas fabriqués à la main ont été ajoutés :

- **Cas adverse 1** — un extrait réel du contrat, mais associé au **mauvais type** de clause ;
- **Cas adverse 2** — un extrait **inventé**, absent du contrat, présenté comme une clause de résiliation.

Les 8 cas ont été posés à chaque outil sous forme de questions oui/non indépendantes, en un seul appel. Aucune préparation propre à un outil n'a été faite après coup : mêmes 8 cas, même attente de réponse.

2.3 Résultat — TypeSafe (Jev), un service payant à l'appel

Cas	Réponse (p vrai)	Verdict	Attendu	Correct
6 clauses réelles	0,95 – 0,97	VRAI	VRAI	✓ ×6
Mauvais type	0,02	FAUX	FAUX	✓
Extrait inventé	0,01	FAUX	FAUX	✓

8/8 corrects, les deux cas fabriqués détectés avec une confiance nette (jamais proche de 0,5). **Coût réel de la vérification complète** : 3 348 jetons en entrée, **0,00014 \$** (0,042 \$ / million de jetons, sortie gratuite) — pour comparaison, une seule analyse de contrat par le modèle actuel coûte environ 100× plus. Service hébergé (API), aucune installation ; nécessite une clé et sort les données du poste (politique de confidentialité et absence d'entraînement sur les données client publiées par l'éditeur, à vérifier avant tout usage réel). Preuves : `preuves\2026-09-23-pilote-typesafe-script.py` , `...-sortie.txt` .

2.4 Résultat — Laya, un modèle local et gratuit

Même protocole, exécuté localement (WSL, aucun réseau après le premier téléchargement du modèle) :

Cas	Réponse (p vrai)	Verdict	Attendu	Correct
6 clauses réelles	0,66 – 0,99	VRAI	VRAI	✓ ×6

Cas	Réponse (p vrai)	Verdict	Attendu	Correct
Mauvais type	0,21	FAUX	FAUX	✓
Extrait inventé	0,99	VRAI	FAUX	✗ raté

7/8 corrects — et le cas raté est le plus important des deux : Laya a donné 0,99 de confiance à un extrait qui n'existe nulle part dans le contrat. Pour un outil censé vérifier qu'un texte est cité mot pour mot, laisser passer une citation inventée avec une confiance quasi totale est le défaut précis qu'on cherche à éliminer. Coût : nul (modèle local, poids ouverts) ; temps : 58 s sur ce test (dont le téléchargement du modèle au premier appel — les appels suivants sont documentés à 1–3 s par lot). Preuves : `preuves\2026-09-23-pilote-laya-script.py`, `...-sortie.txt`.

2.5 Comparaison et lecture honnête

	TypeSafe (Jev)	Laya
Résultat sur ce test	8/8	7/8 (le cas le plus critique raté)
Coût par vérification	≈ 0,00014 \$ (payant, mesuré)	0 € (local, poids ouverts)
Confidentialité	Sort du poste (à vérifier)	Reste sur le poste
Déploiement en production	Un appel HTTPS, marche sur un conteneur Linux CI/prod sans rien de plus	Nécessite WSL ou un conteneur Linux avec Python/torch (fonctionne sans WSL hors Windows)
Statut	 testé, non intégré	 testé, non intégré

Ce qu'un seul test de 8 cas ne prouve PAS, dit comme pour tout le reste de ce projet : un échantillon de 8 cas sur un seul contrat ne suffit pas à choisir un outil de production. Les deux cas adverses ont été choisis à la main, volontairement nets — les vraies hallucinations de ce pipeline sont rares (aucune vue de tout le projet ce soir) et souvent plus subtiles qu'une phrase totalement inventée. Le résultat dit : le mécanisme fonctionne chez l'un, et un vrai défaut existe chez l'autre sur exactement le cas qui compte — il ne dit pas lequel des deux gagnerait sur un échantillon plus large ni sur les hallucinations réelles du pipeline.

3. Si le formateur valide d'aller plus loin

Aucune décision n'est prise ici — ce sont des options, pas un plan engagé :

- Ne rien faire** — aucune hallucination n'a été observée dans le pipeline actuel sur les 12 contrats de référence ni sur les cas de production capturés à ce jour ; le gain resterait défensif, pas correctif.
- Ajouter une brique de vérification** (probablement TypeSafe, sur le résultat 8/8) — une étape après l'extraction, avant le calcul du score de confiance final, avec sa propre mesure sur un échantillon plus large (les 12 contrats + les cas de production capturés), son propre test TDD, son propre passage par le gate — exactement le protocole suivi pour chaque brique de ce soir.
- Réduire le coût du modèle actuel** (question posée en cours de projet) — **tranchée depuis** : aux tarifs catalogue entrée/sortie (\$1.3), le coût est de 0,033 € par analyse, sous la cible ; un modèle moins cher (`gpt-4.1-mini`, essayé sur le vrai gate : qualité parfaite sur l'essai) resterait une option, par une nouvelle version et son gate.

Glossaire

- Hallucination** : ici, un extrait de clause présenté comme réel qui ne figure pas mot pour mot dans le contrat, ou un type de clause associé à tort à un extrait qui ne le décrit pas.

- **Vérification indépendante** : un second jugement, produit par un système distinct de celui qui a rédigé la réponse — casse la circularité d'un modèle qui juge sa propre production.
- **P95** : la valeur sous laquelle se trouvent 95 % des mesures — montre la traîne que la moyenne cache.
- **Signature (de dérive)** : la médiane du score et la part de scores bas, mesurées sur des analyses saines réelles — la référence à laquelle la production est comparée pour détecter une dérive.